

The Human and the Machine

Chapters 24–26. Teams, process and ramp-up; machine learning and language models; and the road ahead.

Twenty-three chapters built the machinery. These three are about what points it — an organisation of people, a machine that writes plausible code, and the one question neither of them can be relieved of.

What you will be able to do

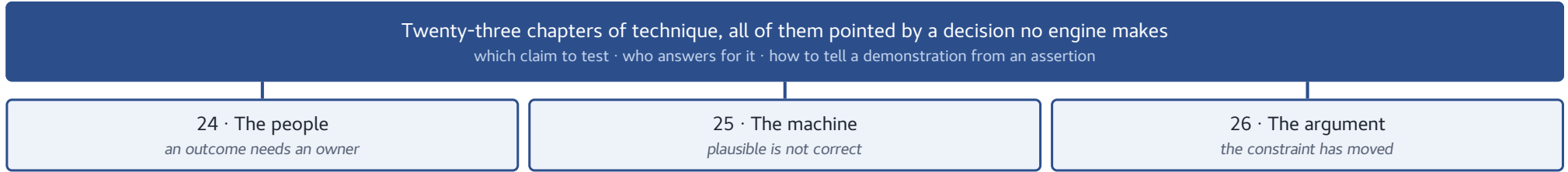
- State a role's charge as an *outcome about the design*, and hear when it has been stated as an artifact instead.
- Run each review rite for what it is actually for — and name the specific way each one dies.
- Report a maturity stage the only way that means anything: *with its open checklist lines*.
- Read a capability claim about a generated artifact — benchmark, pass criterion, baseline, underlying model — *before* reading its percentage.
- Say which constraint binds on *your* block, and what would still be true if every engine got a thousand times better.

THE REGISTER SHIFT

Parts I–VI asked whether the evidence was enough, and built the instruments that produce it. Part VII asks who decided what the instruments were pointed at — and whether that decision can be delegated, to a process, to a junior engineer, or to a machine. It is the least mechanical material in the book, which makes it *more* exposed to confident folklore, not less.

SAY Watch the hedges on these three chapters. Chapter 24 marks its own judgments as the author's read in six places; this deck keeps that marking visible, in amber, on every slide that carries one.

Three chapters, one question: who decides what to ask?



- CH 24

Ownership stated as an **artifact** is discharged by the artifact being good. Ownership stated as an **outcome** is discharged only by evidence about the design — and after an escape, only the second one is still standing.
- CH 25

A generated artifact is **plausible by construction**, because plausibility is what the technology optimises. Read every capability claim through its pass criterion, and never let a model-judged score stand in for a mechanically checked one.
- CH 26

The binding constraint has migrated — cycles, then stimulus, then checking, now **knowing what to check**. Each generation of tooling dissolved one constraint and exposed the next.

SAY *The three chapters answer the same question in three registers: an organisation answers it by assigning it, a machine cannot answer it at all, and the historical read says it is now the whole game.*
HEDGE *Chapter 26 states outright that the four-stage migration is the book's argument, not the corpus's — slide 37 carries that.*

24

Teams, Process and Ramp-up

How roles are drawn, how reviews are run, how progress is talked about, how people are brought into the discipline — and the point past which process substitutes for thought.

Everybody did their job. The outcome had no owner.

Week 41 of the reference SoC. A system-level test running a quantized network on the neural accelerator, with both DMA channels streaming, produces **one wrong output tensor in roughly four hundred inferences**. The mechanism: the accelerator's weight prefetcher wraps its buffer pointer one entry early when the weight stream ends mid-beat under DMA contention. It reproduces only at 2-bit weight precision, only with a degenerate tensor geometry, and only when the other channel is moving traffic. **The block signed off in week 33 at 100% statement and branch coverage.**

THE POST-MORTEM, AND WHY NOBODY IS WRONG IN IT

The accelerator's verifier: memory contention is a system-level concern — her environment drove the accelerator's ports directly, as the plan said, and she closed every quantization row. The SoC integration verifier: quantization corners are a block-level concern — his job was connectivity and system traffic, with no visibility into weight-buffer geometry. The DMA verifier was not invited, because the DMA was not at fault. **Every artifact in the project had a named owner. The outcome the escape falsified — *is the accelerator demonstrably correct while something else is competing for memory?* — had none.**

SAY *Nobody in that room can be told to do better next time, which is exactly the diagnosis. This is an ownership defect, not a diligence defect, and it is invisible until an escape reveals it.*

What is a role answerable *for*?

OWNERSHIP BY ARTIFACT — THE DEFAULT

“She owns the block environment. He owns the SoC environment. That one owns the coverage model.” Administratively convenient, maps cleanly onto repository directories, and **arrives without anyone deciding it**. Discharged by the artifact being good.

OWNERSHIP BY OUTCOME

Not “owns the accelerator testbench” but “**owns whether the accelerator is demonstrably correct under every condition the system can present to it**”. Discharged only by evidence about the design.

This is not a rhetorical upgrade of the same sentence. The two formulations *disagree* about who is at fault when the environment is excellent and the answer is still unknown. The book’s vocabulary already contains the move one level down: an RTL designer’s responsibility is not to write RTL — that is a means to an end — but to produce a working design, which is why the whole design team, not the verification team alone, contributes to the plan and answers for its completeness. Applied to verification roles, “**my rows are closed**” **stops being a defence and becomes an admission**: closed rows are an artifact statement.

AUTHOR’S READ THE TEST THAT SEPARATES THEM

Ask an owner to state **what would have to be true for their answer to be wrong**. An artifact owner describes a bug in the artifact; an outcome owner describes a condition of the design they have not yet reached. A project that cannot get the second answer for a feature does not have an owner for it — it has a directory.

SAY The chapter flags that test as its own judgment, not a published rule — so teach it as a habit you are recommending, not as a finding. And hold on to its shape: slide 44 asks the same question about a whole sign-off.

Boundaries do not fail in the middle of a block. They fail at three seams.

SEAM	WHAT ACTUALLY GOES WRONG	WHAT CLOSES IT
Block → system	At integration each block <i>generally</i> arrives from its own team, and the SoC verification team must debug and analyse the assembled system without knowledge of the IP internals — each block is a black box to that team, which has neither the time nor the background to understand the inner workings of several complex IPs at once. Structural, not a staffing failure.	Name an owner for each <i>cross-block outcome</i> — “the accelerator under memory contention”, “the flagship’s DMA under a coherent writeback” — and put that owner on the <i>block</i> side, where the internals are understood, obliged to state what system conditions the block environment did <i>not</i> present.
Design → verification	Not a wall at all. Design engineers spent, on average, 49% of their time on verification activities in 2024 — half a designer’s working life is already inside this discipline.	Stop drawing it as a hand-over. A boundary drawn that way produces the familiar pathology: design-side checks written but never regressed, because they belong to no suite.
Feature → nobody	Rows whose evidence is expected from somewhere else — a protocol checked by purchased verification IP, a corner covered by the emulation run, a mode “the software team exercises”.	An assumption is a row whose owner is a sentence. Give it a person or delete it.

SAY The first row is the one people argue with. It is not a criticism of integration engineers — it is a fact about what any integration team can know, which is why the fix moves the owner rather than the workload.

The 49% in row 2: 2024 Wilson Research Group IC/ASIC functional verification trend report, commissioned by Siemens EDA — [R2, p. 7]. Double-blind study, ±4% at 95% confidence.

Adopting an artifact can transfer an outcome nobody re-assigned

SCENARIO • THE VIDEO CODEC'S REFERENCE C MODEL

The codec's checking rests on a bit-exact C model. **The design team wrote it first**, to explore the entropy-coding trade-offs, and it is excellent — faster and better tested than anything the verification team would have produced in the time. So verification adopts it as its checking reference and compares the RTL against it frame by frame. Ownership by artifact says this is efficient: design owns the model, verification owns the comparison, nobody duplicates work.

APPROACH

Keep the model, but be precise about what it is — **a transfer function written by the design team, not a golden model** — and move the *outcome*, “the codec's output is what the standard requires”, to the verification side. Discharge it with evidence the C model cannot produce: conformance streams from the standards body decoded against both, an independent reader written from the specification for the syntax layer, and a written record of every place the C model was tuned to match the RTL rather than the specification.

WHY

A model written by the design team encodes **the design team's reading of the specification**, so it is not the independent second interpretation a reference model has to be. Comparing RTL against it reconverges on that reading, and **the comparison can stay green on a misread clause forever**. The artifact is fine. What was wrong is that adopting it silently transferred an outcome nobody re-assigned.

SAY Note the shape of the fix: nothing is thrown away and nobody is blamed. The change is one sentence about who answers for a claim — which is what makes it cheap enough to actually do.

Four rites, four different purposes — and four different ways of dying

RITE	WHAT IT TESTS	HOW IT DEGENERATES
Plan review	Completeness — a property no single perspective can check, which is why its roster deliberately reaches outside the project team.	Everyone present has a stake in the plan being adequate, so it becomes a <i>signing ceremony</i> whose output is a date rather than a list of holes.
Code review of the testbench	Fidelity between intent and implementation — discrepancies no automated tool can find. The declared goal is explicitly <i>not</i> to ridicule the author.	Into style enforcement — the part a tool should already be doing — and it stops asking the question that needs a human: <i>does this environment implement the plan?</i>
Coverage-model review	Whether the measurement is honest. A coverage model is a reflection of the plan, and there is no automated way to establish that either is complete.	When the only attendees are the model's author and the lead who wants it closed. The project acquires a green number certifying <i>stimulus</i> rather than behaviour.
Sign-off review	Judges assembled evidence and accepts what is left over.	It arrives with no room to act: a hole found here is a risk to accept, not a defect to fix. Which is why the three above have to be real.

THE VERIFICATION-SPECIFIC REASON CODE REVIEW MATTERS – IT HAS NO EQUIVALENT ON THE DESIGN SIDE

A testbench routinely acquires temporary structures that **bypass large sections of a testcase** to speed up debugging, and *nothing fails* when those are left in — so the environment quietly stops implementing the tests it claims to contain. Peer review against the plan, plus review of the simulation log to confirm the run followed the specification it claims to execute, is the redundancy that catches it.

SAY One roster constraint: the block's designer must be in the coverage-model review, because whether a cell is reachable by construction is a question about the RTL.

Reviews fail politely far more often than they fail rudely

AUTHOR'S READ

Three of the four mechanics below are the chapter's own judgment, offered because they are cheap and *their absence is diagnostic*. Only the fourth is drawn from the corpus.

- **Circulate the artifact before the meeting**, and open by asking whether people read it. A review where the author narrates the document aloud has already failed — narration is a presentation and the reviewers are an audience.
- **Someone chairs who is not the author**, so the author can answer questions instead of managing the clock.
- **Every finding leaves as an action** with a name and a date, closed *before* the next gate rather than at it.
- **Attach the review record to the object it judged**. Records linked directly to plan objects make the history retrievable during audits and quality reviews — the difference between a review that accumulates into institutional memory and one that evaporates into a meeting-notes folder.

A FIFTH MECHANIC, ABOUT TONE RATHER THAN PROCEDURE

Demand clear justification of how each specification point is verified. Accept that the justification may be complex and needs thoughtful review. And **do not tolerate, or encourage, hand-waving**.

THE FAILURE THAT IS INVISIBLE FROM INSIDE THE MEETING

Where a review has to span two levels of abstraction — a formal property set against a coverage model, say — the mismatch makes the review process much harder, a deep dive into individual items is *often* required, and the time for it has to be allowed for. **A one-hour slot for something needing a half-day does not produce a bad judgement. It produces no judgement, delivered on time.**

SAY *And the way every rite dies under schedule pressure is the same: the review happens after the decision it was meant to inform, so its findings can only be recorded as risks.*

An assumption naming a team is not closed until someone from that team names the row

SCENARIO • THE COHERENT HUB'S PLAN REVIEW

The hub's own rows are dense and well argued — MESI state transitions, directory occupancy, the livelock and starvation cases. In the assumptions section sits one line: “IO-coherent port behaviour under snoop-in is covered by the accelerator teams.” Two of the three accelerator plans contain the mirror-image sentence: “coherency is handled by the hub.” **Nobody wrote a row.**

THE COUNTERPARTY RULE

An assumption naming a team is not closed until **someone from that team has read it in this review and named the row that discharges it**. Here that produces two new rows on the hub's plan — snoop-in during an in-flight accelerator write, and snoop-in to a line the accelerator has just released — and one row on each mirroring accelerator plan.

Why: both sentences are individually reasonable and jointly fatal. Unspoken and undocumented assumptions are how design features fall through the cracks — which is why every assumption is reviewed with the designers face to face, *usually* over several rounds before alignment.

THE SAME DISCIPLINE, ONE RITE OVER – THE CROSSBAR'S COVERAGE-MODEL REVIEW

A proposed cross would stress ID handling at a subordinate port: two managers presenting the *same* manager-side ID with writes in flight to one subordinate. The designer kills it in a sentence. The interconnect appends source-identifying bits toward the subordinate, so the subordinate-side ID is **7 bits** — the 4-bit manager ID plus the 3 bits needed to distinguish 5 manager ports — and the two managers arrive with *different* IDs. **The cell can never be hit, by construction.** The row is not deleted, it is **replaced**, and the minutes record why.

SAY An unreachable cover point does not fail loudly. It sits at zero, absorbs stimulus effort when time is shortest, and is eventually excluded by an engineer who no longer remembers whether it was impossible or merely unreached — while the hazard it represented stays unmodelled.

Three conversations that change shape — from confidence to evidence

THE QUESTION	ANSWERED WITHOUT STAGES	ANSWERED WITH THEM
“Are you on track?”	A percentage and a tone of voice, neither of which the asker can audit. A plan carrying too little detail communicates poorly with other teams, leaves them confused about what has <i>actually</i> been verified, costs the verification team their trust, and pushes everyone into proxies for progress — coverage percentages, test counts.	A stage claim is auditable in an hour by someone who does not work on the block , because every line of a stage checklist is a demonstration, not an estimate. The conversation stops being about whether the lead is an optimist.
“Can I build on your block?”	A per-team document, and a document-merge exercise that substitutes for agreement.	A <i>comparable</i> answer: stage 2 means the same thing for the crossbar and the DMA and the core. Where one plan is held centrally and every team's view of it stays in sync, that single plan becomes the verification contract across sites .
“Who needs help?” AUTHOR'S READ	“85%” asks for nothing, because there is nothing in the number a lead can act on.	“Stage 2, four checklist lines open, two of them blocked on the formal proof of the ordering rules” asks for a specific intervention . On the chapter's own read, this is the conversation that pays for the whole apparatus.

SAY Stages also make it socially possible to say not yet without saying we are behind — the stage is a state, not a verdict. Teams that cannot say “not yet” say “nearly” instead, for months.

A stage claimed rather than demonstrated has become a status word

The signature distinguishes stage inflation from ordinary optimism: **the stage arrives *without* its checklist**. And a status word can be assigned by whoever wants it assigned — which is generally not the team. The conversational countermeasure belongs to every engineer, not just to leads: **never report a stage without the open lines**. “Stage 2, three lines open” is a complete sentence. “Stage 2” is a slogan.

SCENARIO • THE SENSOR HUB ARRIVES LABELLED “VERIFICATION MATURE”

Delivered by a partner team as a pre-verified block for the reference SoC's always-on island. **The integration lead does not argue about the claim; he asks for the stage-2 checklist with its tick marks**. It comes back with the stimulus, checker and regression lines ticked and three lines blank: retention behaviour across the hub's ultra-low-power modes, asynchronous wake events crossing into the system domain, and the functional coverage model for low-power mode transitions.

APPROACH

Accept the block **at stage 2 for the functions the checklist covers**. The three open lines become rows on the SoC plan, with the partner team named as owner and a date. Integration proceeds; the low-power sequencing tests are gated on those rows, not on goodwill.

WHY

The claim “verification mature” was **not false, it was unqualified**. The checklist converted a trust question — *do we believe this team?* — into an evidence question both sides read the same way, and it did so **without anyone having to allege anything**. That is the whole social function of the vocabulary.

SAY One caution worth repeating in any status meeting: a stage describes the maturity of the verification, not the cleanliness of the design. A block at stage 2 with fifty open bugs is reporting that the machinery which finds bugs is working — which is exactly what stage 2 claims.

Verification engineers *often* arrive from design — and the reflexes come with them

WHAT TRANSFERS, AND IT IS A LOT

Reading RTL at speed, which decides how fast a failure gets localized. Micro-architectural intuition about where designs are fragile: which counters wrap, which FIFOs have a subtle almost-full, which handshakes have a reset hole. Knowing how the block was built. And standing with the design team when a verifier has to say a failure is a design bug.

The mindset is *usually* reached only through years of experience, mentoring and lessons learned the hard way. So the transition is **slow** — a two-day tool course is not a ramp-up — and it is **taught, not read**, so a wiki is not one either.

Intel: 85 engineers trained on formal property verification in two weeks, then four weeks of onsite consultancy — *Making Formal Property Verification Mainstream: An Intel Graphics Experience*, DVCon — [D18, p. 1]. Self-reported; a ramp onto one technique, not the mindset.

What works against them. Reflexes, not deficiencies — each correct in the previous job, and several make a testbench look healthier while it checks less.

Explaining before reproducing **AUTHOR'S READ**

Interoperating, not stressing

“Invalid” as a stimulus filter

A testbench that recovers

Debugging in waveforms

Staying zoomed in

A verifier's first obligation is a failure that replays on demand. Hypothesis-first gives a diagnosis with no artifact behind it.

The more tolerant the testbench, the less it checks. Recognizable form: *clock snooping*. Fewer failing tests, faster bring-up, masked clocking bugs.

What matters is whether the scenario *can occur* — and if it can, the recovery from it is a feature.

A testbench is closer to a mousetrap: once it has caught something, effort spent keeping it going is effort in the wrong place.

A verification component's work happens in zero time and in dynamic data structures. The *log* is where it gets debugged.

The work requires switching deliberately between zoomed-in tasks and zoomed-out ones — where effort buys the most checking.

Reproduce, extend, own, break — and the order is the point

AUTHOR'S READ The ladder's shape is the chapter's own; its load-bearing components are not. Four assignments for a designer joining DMA verification, each with an exit criterion stated as a *demonstration* rather than a duration.



- 1 Replay 3 archived failing seeds from their run manifests; one bug report per failure at *mechanism* level. **Done when** all 3 reproduce on demand and the mentor cannot tell the new reports from the originals.
- 2 Add the per-channel error-report cross-check to the scoreboard, treating the DMA's own error report as an *output to be checked*, never as ground truth. **Done when** peer review is passed, the check fires on a deliberately broken error-report path, and is silent on 200 clean seeds.
- 3 DMA-F06a (2D legalization at the 4 KB boundary) end to end: implement `cg_2d_4kb`, drive it to closure, defend the exclusion. **Done when** the six excluded cells are a waiver naming `c_solver_budget` — not written off as geometry — with owner and expiry.
- 4 The *mentor* injects 3 undisclosed bugs into a private RTL copy; run the full regression; report which were caught, by what, and how long each took to localize. **Done when** any bug the environment missed becomes a plan row.

SAY Two things the ladder deliberately does not do: it does not open with a course on the methodology library, because a newcomer with no failure to chase has no reason to remember any of it; and it does not open with new code, because **the first artifact they produce is a bug report** — the report, not the testbench, is what the rest of the project consumes. **ALSO** Assignment 3 gets four weeks because it teaches judgment: the six cells are dead because the solver budget caps a row at 2048 bytes — 256 beats on the 64-bit path — a team decision about solve time, not a property of the design. Around the ladder: a named champion per cluster, escalation to a central expertise team, and a library of verified checkers to adapt.

Process is the accumulated answer to “how do we not make that mistake again”

INSTRUMENT	WHAT IT ACTUALLY BUYS – WHICH IS USUALLY NARROWER THAN TEAMS CLAIM
Templates	Consistency across authors. A plan template is called <i>crucial</i> because a large project has many plan authors feeding one guide. The real product is not the section headings: it is that ten plans can be read by one reviewer without ten context switches.
Coding standards	One published hierarchy separates <i>rules</i> — which must be followed, because breaking them costs the productivity the methodology exists to provide — from <i>recommendations</i> , where the detail is often explicitly unimportant (a naming convention is the example given), but adherence within a team is strongly recommended. For a large class of standards the content is arbitrary and the agreement is everything.
Libraries and checklists	A lower entry barrier and a memory. Cookie-cutter property templates for the modules that recur — arbiters, state machines, standard interfaces — so a novice meets a filled-in starting point rather than a blank file.
A calendar slot	Stated flatly: unless the practice is integrated into mainstream deliverables, it will not be exercised despite its benefits — which is why check-points analogous to the existing milestones were introduced. <i>A technique not on somebody's deliverable list does not happen.</i>

SAY What it costs is adherence and discipline, charged continuously rather than once. Wholesale adoption of a full methodology **may** simply be unaffordable for a real project with real people and schedules — an argument for adopting individual elements incrementally, not for adopting none. And the alternative is not free either: best-known methods simply not being in use is estimated to cost on the order of 20% of productivity on its own. AND One coding standard genuinely has content: comments written for a reader who knows the language but not the design, and must maintain it without the original designer's help.

The line is not good process versus bad. It is auditable claim versus laundered judgement.

It cannot decide whether its own line is true	“Functional coverage model implemented and reporting in nightly regression” is objectively true or false; anyone can go and look. “Coverage is <i>adequate</i> for this feature” is a judgement wearing a tick-box. Every line of a gating checklist must name an artifact a skeptic could regenerate — and a line that cannot be written that way belongs in a review discussion instead.
It cannot see what is missing from the plan	Every line audits a question somebody already thought to ask. This deck's opening escape would have passed a stage-3 checklist unblemished , because no line asked about the block under memory contention. Which is why escape analysis exists, and why its output is <i>new rows</i> .
It cannot make a review honest	It converts a judgement into a tick — useful for recall, useless against motive. If the room wants the gate passed, the checklist supplies a format for passing it.
It cannot narrow the spread that matters most	Individual productivity across engineers ranges up to tenfold. Process compresses variance in the mechanical parts — how a bug is filed, when a regression runs — and does approximately nothing to which corner someone thinks to chase, or whether an odd waveform is investigated at 5 p.m. on a Friday.

THE TELL — A CHECKLIST LINE CONTAINING THE WORD *ADEQUATE*, *SUFFICIENT* OR *ACCEPTABLE*

Split it. The auditable half stays on the checklist; the judgement becomes a named review item with a threshold agreed in advance.

The tenfold spread: Oracle Labs, *I created the Verification Gap*, DVCon U.S. 2015 — [D16, slide 7]. Practitioners' own figure, not a controlled study.

SAY *The worked case is the radar front-end's precision sign-off: an inherited line, “quantization error characterized against the reference model across the operating range”, ticked on every project since. The characterization ran and the report exists — and nobody asks whether that error is acceptable for the CFAR stage downstream, whose false-alarm rate is what the customer actually buys.*

AND *There is a point past which process stops paying, with a recognizable shape: too much detail too soon, where the result communicates worse, not better.*

25

AI and ML in Verification

What has been *demonstrated*, what is *claimed*, and the discipline of telling them apart. Every figure below carries its year, its benchmark, its pass criterion, and — where it matters — the model underneath it.

Both numbers are real. Only one is evidence about the thing being proposed.

PROPOSAL 1 • THE VIDEO CODEC'S TRIAGE CLASSIFIER

Trained on eighteen months of failure logs; it sorts each morning's failures into buckets and names a probable cause. **On a held-back month of history it agreed with the human triage engineer 88% of the time.**

Measured against a *baseline the team owns*: yesterday's triage decisions, made by a person, written down, and available to disagree with. It can be re-measured next quarter on next quarter's failures — and if agreement falls, the team will see it fall.

PROPOSAL 2 • THE CRYPTO ENGINE'S ASSERTION PILOT

A language model was given the engine's specification chapter and produced forty SystemVerilog assertions in about twenty minutes. **Thirty-eight compile. Thirty-six pass a formal proof against the current RTL.**

This measures agreement between a generated property and *the design it was generated to check* — precisely what an assertion must not be judged by. A property whose antecedent this design never satisfies passes without ever having been evaluated. **Thirty-six passing proofs is a statement about compilation and consistency. It is not a statement about intent** — and intent is the only thing a specification-derived property exists to capture.

The field's own reviewers say as much. A 2025 review of machine learning for micro-electronic design verification observes that although these techniques have been proposed since the late 1990s, *many* never reached mainstream adoption — and names three reasons that are about **evidence rather than capability**: withheld code and data make results hard to reproduce; techniques are evaluated on simple designs against random stimulus and little else; and research rarely asks whether a method generalises past the one application it was tried on.

SAY Two risks attend a subject like this — that it ages into embarrassment, and that an engineer promised a great deal and shown less reads it cynically and stops. Both are managed the same way: separate what has been demonstrated from what is claimed, and date everything. A vendor demonstration is not evidence.

The least glamorous techniques here are the most deployed — and each optimises something narrower than the goal

Three applications have a genuine track record: **steering stimulus toward uncovered behaviour**, **sorting regression failures**, and **choosing which tests to run**. Their published results span two decades rather than two years. The value in reading them now is not the numbers — it is the *shape* of the numbers.

THE FOUNDING COVERAGE-DIRECTED RESULT, 2003

A Bayesian network trained to direct stimulus at the storage control element of an **IBM z-series** system. The coverage model had five attributes with a cross product of **13,888** combinations, of which **1,968** were legal coverage tasks. Training used **160** test-cases, each taking *more than* thirty minutes to run, and covered **1,745** of them; **223** remained.

The trained network covered all 223 within **250** further test-cases — where an expert-written directive file covered *two-thirds* of them after more than **400**.

READ THE SECOND HALF OF THAT BEFORE THE FIRST

The baseline was a human expert's tuned directive file, not uniform random stimulus. That is rare, and it is what makes the result mean something. The 2025 review notes that *random* is the most common baseline in this literature, and cautions that beating random says nothing about whether a technique generalises.

AND THE COST IS IN THE SAME FIGURES

160 training cases at more than thirty minutes each is **more than eighty hours of simulation before the model produced its first useful directive** — on top of building a coverage model precise enough to learn from.

SAY The structural point from Chapter 6 is what makes all of this behave the way it does: every such technique learns the mapping from generator directives to achieved coverage and inverts it, which means it optimises the derivative rather than the goal.

Cross product, legal tasks and both learning curves: Fine and Ziv, IBM Research Haifa, *Coverage Directed Test Generation for Functional Verification using Bayesian Networks*, DAC 2003 — [P16, pp. 5–6]. Self-reported. The per-test runtime, and the eighty-hour figure derived from it, are not among the figures checked on those pages.

It buys the hard bins. It does not buy closure.

YEAR	WHAT WAS REPORTED	WHAT IT DOES NOT SAY
2012	A review of the whole literature found a limitation common to every approach it surveyed: none could consistently cover all the tasks in a coverage model.	Its adoption barrier has aged well: the <i>average</i> verification engineer is not an expert in these techniques and cannot tune one, so a method needing customisation will not be supported. An approach fit for industry must produce results engineers can easily interpret.
2023	Deep reinforcement learning on a compression encoder: an agent reached 133 of 136 hard functional bins, where uniform-random stimulus plateaued at 28 .	Closure required merging the coverage of two separate training runs , because neither alone hit every bin — a 500-episode run reached 133, a 750-episode run reached 127, and the second run's bins were <i>not a subset</i> of the first's. A plan that had promised closure would have been wrong.
2024 via a 2025 survey	A system given the coverage report and the design under test and asked to propose stimulus beats random testing on simple and medium designs.	It degrades above roughly sixty-four states — which the survey reads as an argument for <i>combining</i> language models with structural approaches rather than replacing them.

A GOOD TARGET – THE GPU'S SHADER CLUSTERS

Four clusters, a scheduler, and a coverage model whose interesting bins are combinations of occupancy, divergence and memory-request pattern that nobody wants to enumerate by hand. The fit is good because **the space is large, the legal directives are many, and a bin is cheap to check.**

A POOR ONE – THE CRYPTO ENGINE'S KEY LADDER

The interesting states are few, reachable by a directed test in an afternoon, and defined by **a security argument that no coverage bin expresses.**

Two halves of one problem, on one feature set — and they behave completely differently

TASK · ONE 2018 STUDY, 616
FEATURES FROM SEMI-STRUCTURED
SIMULATION LOGS

	REPORTED	WHAT IT NEEDS, AND WHAT THAT IMPLIES
Unsupervised clustering of failures	0.543 and 0.593 adjusted mutual information across three algorithms, against an ideal of 1.0	No labels — and it is weak.
Supervised classification of the root cause	90.7% accuracy, F1 0.913 with a random forest	Strong — and it needs a <i>labelled history</i> : somebody must already have looked at hundreds of failures and written down what caused each.

THE GAP POINTS THE WRONG WAY FOR ANYONE HOPING TO BUY THEIR WAY OUT OF TRIAGE

The technique works best **where a team already has years of curated triage decisions**, and worst on a new project or a block's first tape-out — which is where triage hurts most. The same survey ladders the problem into three — cluster failures by cause, classify the cause, suggest the fix — and reports that *most* research addressed the first two, with **no results available on the third**.

THE ECONOMICS

That survey also reports a *typical* machine-learning project spending **up to 70%** of the team's effort on preparing data. On the video codec's nightly that is not hypothetical: the failure population is dominated by golden-model mismatches whose useful label is *a frame region and a coding tool* — and neither is in the log unless somebody puts it there.

WHICH IS WHY CHAPTER 12'S DISCIPLINE IS WHAT MAKES THIS AFFORDABLE

The triage engineer's confirmation is the training data — and only if that confirmation is recorded in a form a model can read. A rota that closes buckets in a chat thread has spent the labour and thrown away the label.

A compressed regression is a change-detection instrument, not a bug-finding one

A RANKING MODEL OPTIMISES

Time-to-first-failure over a *known* suite — it reorders existing test-and-seed pairs so the runs most likely to fail go first.

A SELECTION MODEL OPTIMISES

Coverage per simulation-second — discarding runs whose contribution it predicts is redundant.

NEITHER OPTIMISES

The discovery of new behaviour. The industrial study behind the ranking figures says so in its own words: a ranked regression *simulates no new scenarios* and is *not recommended* for exploring the random space or exposing new bugs.

That is a constraint on where the technique can sit, not a caveat to file away. A ranked or compressed regression answers “*did this commit break something the suite already knew how to catch?*” quickly and cheaply. **A program that replaces its full random regression with a compressed one has quietly converted its main source of new bugs into a fast regression check.**

THE HONEST DEPLOYMENT KEEPS BOTH INSTRUMENTS

A compressed suite per commit, *and* a full sweep with fresh seeds on a slower cadence — whose yield is tracked as a signal in its own right. The failure mode is quiet and arithmetic: regression runtime halves, and the bug discovery rate follows it down where nobody is plotting it.

SAY This is the one entry on the adopt-now list that comes with a mandatory companion. If a team takes the compression and drops the sweep, the technique has made the programme worse while every dashboard improved.

Every technique on the last four slides shares one failure — and it is why published results do not survive contact with a programme

A model trained on one design's coverage response, one quarter's failure logs, or one release's regression outcomes is a **claim that tomorrow resembles yesterday**. The 2025 review names this directly: an example that was novel becomes unremarkable once more examples have been seen, so the model needs regular retraining to stay relevant — and worse, once deployed, **the learner shapes the examples it will later be retrained on**, which *can* suppress exploration and make performance *decrease* over time. The same review observes that research usually frames verification as a one-time learning problem while industrial designs change continuously, and warns that if substantial effort goes into crafting fitness or reward functions, the technique has *merely shifted* effort from writing tests to tuning models.

CONSEQUENCE 1 • RETRAINING IS A TASK, NOT A MAINTENANCE DETAIL

It is named, owned, scheduled, and has a cost. **A triage classifier 90% accurate at tape-out and 60% accurate two derivatives later is worse than none**, because the rota has stopped checking it. Give it a measured agreement rate, re-measure on a fixed cadence against human decisions, and publish the number where the people relying on it can see it.

CONSEQUENCE 2 • A SEVERITY DISTINCTION THE WORD “MODEL” HIDES

A **stimulus** model that goes stale *mis-steers*: you get worse coverage per run, and the coverage report tells you. A generated **checker** that goes stale is a *bad oracle* — it decides whether an observed response is correct — and **a bad oracle does not announce itself in any report**.

SAY That second box is the hinge of the whole chapter. Everything from here on is about generated checkers, because they are the artifacts whose failure is silent.

The number is 89%. Its four qualifiers are what it means.

The result	A 2024 framework that reads a full natural-language specification and emits SystemVerilog assertions reported 89% of 56 generated properties correct both syntactically and functionally.
... on what	One serial-bus design, 23 signals.
... where correct means	The property passed a formal proof against known-good RTL .
... after what	After human engineers had removed the generations referencing signals the tool could not map . Those never entered the denominator.

THE ABLATION — REPORTED BY THE FRAMEWORK'S OWN AUTHORS AS THEIR BASELINE

Given the same specification files, the underlying general-purpose model *on its own* produced **71** candidate properties, of which **23** were syntactically correct and **6** passed the proof — **8%** by the same measure that gave the full framework 89%.

So the engineering *around* the model — splitting the specification, mapping signal names, retrieving the right passage per property — is doing most of the work. **It is also the part that does not transfer for free to a specification written in a different house style.**

THE SCOPE — WHAT THE GENERATOR COULD NOT REACH

Of the three property classes the framework generates, **connectivity properties could be produced only for control signals**, because the connectivity of data signals depends on internal RTL structure the specification does not describe. And **function properties only where the specification detailed the register behaviour**.

The reachable subset of a specification is narrower than the specification — and a generator will not tell you which rows of your plan it left untouched.

SAY Ask the room which of the four qualifiers they would have heard in a hallway version of this result. Usually none of them, and the fourth one — the hand-removal before scoring — is the one that changes the number most.

A framework built for one artifact class does not transfer to another

A 2025 study attacks a different target: **protocol formal specifications — documents with timing diagrams**. Its flow is two-stage: generate candidate properties from a grammar, check each against the timing diagram with a SAT solver, then use a language model to filter the survivors against the textual description. **The SAT stage retains between 5% and 22% of the candidates in every case** — the clearest available measure of how aggressively a mechanical filter can thin a candidate set before anyone reads it. Across six open-source on-chip protocols, the extracted properties covered **53 of 58** manually derived target properties — *for every protocol but one, which is excluded from that count*.

THE HEAD-TO-HEAD, AND THE REASON IT IS NOT A CURVE

That paper also runs the specification-driven framework of the previous slide on one of its own protocols. **The two results must not be read as a curve**. They differ in task and in artifact class, and the head-to-head *re-ran that framework itself*, reporting it under two underlying models. On that protocol the two-stage flow covered **all 13** manually derived properties with no syntax errors, while under *one of* those two models the specification-driven framework produced **38** properties of which **3** were judged correct and **2 of the 13** targets covered, with **16** syntax errors and **11** references to signals that do not exist. The conclusion is not that one framework is better than another by some ratio — and **the eleven non-existent signals track the model rather than the framework: under the other model it produces none**.

A TAXONOMY WORTH BORROWING WHOLESALE — IT IS THE VOCABULARY A REVIEWER NEEDS

A generated property may be **correct** (unique or redundant), **incorrect**, **unsure**, or **hallucinated** — invented rather than drawn from the set it was given to filter. One protocol's transaction section produced **four hallucinated properties**.

THE AUTHORS' OWN LIMITS, STATED PLAINLY

The method **depends on unambiguous timing diagrams**. On the complex protocols the target set *does not cover the entire formal specification*. Models remain **poor at reading figures**, often getting a signal's value in a given cycle wrong. And **experienced engineers are still better at inferring properties where the document has no text to work from**.

“Around 40%” — and then read what was counted

The corpus contains one industrial report of an end-to-end agentic formal flow, from **Infineon** in 2025: a lead agent drafts a verification plan in natural language, a dispatcher delegates assertion generation, critic agents evaluate the results, properties are proven in a commercial model checker, counterexamples are analysed, and formal coverage is assessed for sign-off. Its headline is that **the flow does not guarantee a result on every run and works most of the time, at an overall efficacy of around 40%**.

WHAT THE 40% MEASURES

The study's **first-named** indicator is a *success rate*, defined as the number of runs that completed end-to-end formal verification out of the total runs — **a measure of whether the pipeline finished, not of whether the properties it produced were the right ones.**

On this flow those are different questions. The authors list **vacuous passes** in formal properties among the failure modes they observed, alongside syntactically incorrect or semantically irrelevant assertions, assertions referencing signals absent from the specification, and truncation of large RTL against the model's context limit.

WHAT IT WAS MEASURED ON

Fifteen designs across three difficulty tiers, where the *advanced* tier holds a **Booth multiplier, a pipelined adder, a small RISC-V core, a PID controller and a round-robin arbiter.** These are block-scale exercises, and a **40% figure should be read as 40% on designs of that size.**

AND IT IS A FUNCTION OF WHICH MODEL WAS UNDERNEATH

The same flow was run with three general-purpose language models. On the easiest tier they completed **17.78%, 43.19% and 48.77%** of runs end to end; on the hardest, **4.33%, 9.00% and 41.43%.** **The two models within six points of each other on the easiest tier were the 9.00% and the 41.43% on the hardest.** That is not a property of the technique but of the technique *paired with one model.*

SAY The flow's most transferable idea is not in its numbers. When two agents loop on generating and correcting an assertion without converging, it stops them at a threshold of five iterations and asks a human for the correct assertion — a designed hand-off point rather than a fallback. *CAREFUL* Do not draw those six figures as a table of model against tier: the source never says which easiest-tier result belongs to which hardest-tier one, and a table would invent that pairing.

The “around 40%”: Saarthi: *The First AI Formal Verification Engineer*, Infineon Technologies, DVCon 2025 — [D25, p. 12]; the advanced-tier design list and the named failure modes, p. 11. Self-reported. The benchmark designs are exercises, **not Infineon products.** The six per-model completion percentages lie outside the pages checked here.

A score awarded by a language model and a score checked by a machine do not rank the same alternatives the same way

A 2025 system from NVIDIA attacks counterexample debugging structurally: it converts the failure trace into a causal graph, scans for suspicious nodes, explores them agentically, and proposes an RTL fix. On 38 real debugging challenges with human-verified ground-truth fixes it scored 0.956 on best-hypothesis quality, and its fixes resolved the failure 71.1% of the time first attempt, 86.8% within five.

METRIC FAMILY	HOW IT IS PRODUCED	CONFIGURATION	QUALITY	RESOLVED WITHIN FIVE
Hypothesis quality	Scored by a language model comparing the hypothesis against the ground truth	The full system	0.956	86.8%
Pass@k	Measured mechanically — the proposed fix applied to the RTL and the proof re-run in a model checker	One unstructured baseline	0.783	65.8%
The paper is explicit about the difference. Its own baseline table shows the two do not track each other →		Another unstructured baseline	0.474	WORST IN TABLE 81.6%

WHAT THAT MEANS FOR A PROPOSAL YOU ARE HANDED — AND WHAT HAPPENS ON THE HARDER DESIGNS

A model-judged score measures how convincing an explanation reads. It is not a proxy for whether the fix works, and where a study reports only that kind, you have been told how plausible the output is. Taken to two counterexamples from a larger open-source application-class processor, best-hypothesis quality falls to 0.713 and no Pass@k is reported at all: the authors call automated fix generation at that scale future work.

0.956, 71.1% / 86.8% and the 0.783 / 65.8% baseline: FVDebug, NVIDIA, arXiv 2510.15906 — [P22, p. 7]; the 1,918-line LSU failure, p. 8. Self-reported; NVIDIA names no product or design of its own — the harder pair is the open-source CVA6. The 0.474 / 81.6% baseline and the 0.713 fall are outside the checked pages.

Read the fourth column first — it is where most summaries of this field go wrong

CAPABILITY	REPORTED RESULT, YEAR	MEASURED ON	WHAT COUNTED AS SUCCESS	STATED LIMITS
Assertions from a specification document	89% of 56 properties correct, 2024	One serial-bus design, 23 signals	Passing a formal proof against known-good RTL	Unmappable generations removed by hand before scoring; connectivity properties limited to control signals
Properties from a protocol specification with timing diagrams	53 of 58 targets covered, 2025	Five of six open-source protocols; the sixth is outside that count	Covering a manually derived target, after a SAT check and a model filter	SAT retains 5–22% of candidates; on complex protocols the target set is not the whole specification
End-to-end agentic formal verification	17.78–48.77% of runs completed on the easiest tier, 4.33–41.43% on the hardest, 2025	15 block-scale designs, 3 tiers, 3 models	A run that <i>completed the flow</i> — not a property that was correct	Efficacy summarised at around 40%; failure modes include vacuous passes and context truncation
Root-cause hypotheses for a counterexample	0.956 best-hypothesis quality, 2025	38 challenges with verified fixes, one model	Similarity to the ground truth, scored by a language model	Model-judged, and does not order configurations the same way the mechanical measure does
Fixes for a counterexample	71.1% first attempt, 86.8% within five, 2025	The same 38 challenges, the same model	The fix applied to the RTL and the proof re-run	Not reported on the two application-class processor failures; called future work

SAY Rows four and five describe the same paper’s system, twice, on purpose. A row that merged them — root-cause analysis, 0.956 quality and 71.1% fix rate — would be true in both halves and wrong as a whole, because it would hide that one number was awarded by a language model and the other by a model checker. ALSO Two rows this table cannot hold. That framework’s own ablation: 6 of 71 candidates passing without the engineering around the model. And testbench scaffolding and plan drafting, for which this corpus contains no measurement of artifact quality at all — the flows that draft a plan are graded only by downstream outcomes.

Row 3: Saarathi, Infineon Technologies, DVCon 2025 – [D25, pp. 11–12]; its tier percentages lie outside those pages, the “around 40%” summary is on p. 12. Rows 4–5: FVDebug, NVIDIA, arXiv 2510.15906 – [P22, pp. 7–8]; NVIDIA names no design of its own. Both self-reported. Rows 1–2 are not anchored to a named organisation.

Correctness is not the objective function. Plausibility is.

```
// Specification: "While the engine is holding a request
// that the memory path has not accepted, the key slot
// selector must not change."
// GENERATED -- and wrong, in two directions.
property p_keysel_stable_gen;
  @(posedge clk) disable iff (!rst_n)
    (req_valid && !req_ready) |-> $stable(key_sel);
endproperty
a_keysel_stable_gen : assert property (p_keysel_stable_gen);
```

```
// CORRECT -- the obligation lands one tick later.
property p_keysel_stable;
  @(posedge clk) disable iff (!rst_n)
    (req_valid && !req_ready) |-> ##1 $stable(key_sel);
endproperty
a_keysel_stable : assert property (p_keysel_stable);
c_keysel_held : cover property (@(posedge clk)
  disable iff (!rst_n)
    (req_valid && !req_ready)[*2]);
```

The signals are right, the guard is present, the condition is the one the sentence describes, and the operator is the one *stable* suggests. **It is wrong in two directions, from one missing delay.**

1 • IT FORBIDS SOMETHING LEGAL

`$stable` compares the current tick against the previous one, so at the *first* held cycle the property demands that `key_sel` already had its value one cycle *earlier* — before the request existed. A design that legally presents a fresh request with a new key slot right after a previous transfer fails a rule the specification never stated.

2 • IT PERMITS THE DAMAGE

On the *last* held cycle it says nothing about `key_sel` in the following cycle — the one in which the request is finally accepted, **where a change would do actual damage.**

Now ask what a formal proof would say about the first version. It depends entirely on whether this pipeline happens to hold `key_sel` steady for a cycle before a request rises. If a register upstream loads the selector one cycle early, the wrong property is *proven* on RTL that is behaving correctly — and under a pass criterion of *proved against known-good RTL* it is scored correct and enters the suite. **A proof engine's verdict on today's RTL is not a verdict about the property.**

SAY The defect is not invented: a 2025 study of property generation from protocol documents opens with it on a generic valid/ready interface, and notes that earlier machine-learning work made exactly the same mistake.

A generator's mistakes are systematic — so sampling a batch estimates nothing

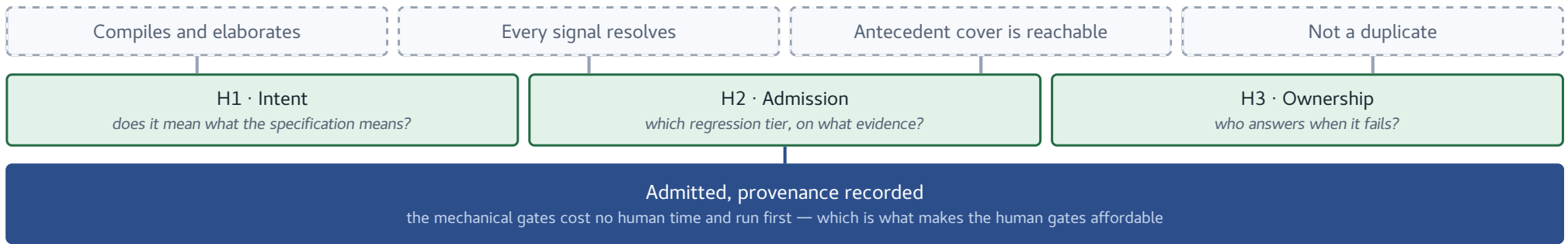
PRACTITIONER JUDGMENT A human writing forty properties makes *idiosyncratic* mistakes, uncorrelated across the batch, so sampling five is defensible: **the sample estimates a rate**. A generator's mistakes are *systematic* — one misreading of one recurring pattern reproduces identically in every property derived from it, so a sample of five either catches the whole class or misses it entirely. The chapter offers this as its own judgment rather than a published result.

WHAT THIS REVIEW CHECKS THAT AN ORDINARY CODE REVIEW NEED NOT	WHY IT CARRIES WEIGHT HERE THAT IT DOES NOT CARRY WHEN THE AUTHOR IS A PERSON
Every referenced signal exists <i>and is the one intended</i>	Generated properties can name signals not in the design — the head-to-head counted eleven in a batch of thirty-eight. Elaboration catches those. It does not catch the <i>wrong-but-existing</i> signal , the common case where ports are similarly named.
The artifact is in a position to fail	Vacuous passes are a named, observed failure mode in industrial use — which promotes Chapter 11's cover-with-every-implication rule from good habit to <i>admission criterion</i> .
The batch, not the sample	Because a generator's errors are correlated, a review covers every artifact in a batch or rejects the batch .

WHICH SETS THE REAL CONSTRAINT ON ADOPTION — AND IT INVERTS THE INTUITION THE TECHNOLOGY INVITES

A verification suite has **collective authority**: engineers believe a passing regression because they believe its checkers were written by someone who understood the design. One careful-looking property encoding the wrong rule spends that credit, and **when the reviewer's scepticism runs out the outcome is not that generated properties get rejected — it is that all properties get skimmed**. So **the limit on throughput is not how fast artifacts can be generated but how fast they can be read**: a flow producing two hundred properties in an afternoon has saved nobody a week unless someone reviews two hundred. *A design constraint on the flow, not an attitude toward the technology.*

“Keep a human in the loop” has no operational content. Name the places instead.



H1 Intent	Owner: whoever owns the plan row it discharges , not whoever ran the generator. Needs the specification sentence beside the artifact, and a <i>demonstration that the artifact can fail</i> . A passing proof is not evidence at this gate . Unowned: a wrong rule enters the suite and inherits its authority.
H2 Admission	Not “does it fail at random?” but “ does it fail on things that are not bugs? ” A generated check earns a gating tier by running clean across a window of known-good history, and <i>that window is the evidence</i> .
H3 Ownership	Named <i>at merge, in the file</i> — not discovered at the first failure. A component nobody wrote is a component nobody can reason about under time pressure . Require the accepting engineer to explain it without regenerating it.

SAY There is a fourth gate the figure does not draw: **waiver authority**. A model's proposed coverage exclusion is still a waiver in Chapter 7's sense, with the same five fields, and its origin does not change who signs it — exclusion is where an optimising system will always push, because to a coverage metric excluding a bin is indistinguishable from covering it. **AND** A fifth place a person belongs is a bound rather than a gate: where two automated stages iterate against each other the loop needs a cap. A 2026 survey names the failure a cap prevents, deadlooping, alongside context loss and tool hallucination.

Two years after tape-out, somebody asks where that property came from

A field failure triggers an escape analysis. Somebody asks how a behaviour was covered; the answer points at a property; the next question is where it came from. **If the honest answer is a model wrote it and someone approved it, the review has nowhere to go.**

ATTACH TO EVERY GENERATED ARTIFACT – IN THE REPOSITORY, NOT IN A WIKI

What produced it	System, version, and the identity and version of the model underneath . One flow varied from 4.33% to 41.43% end-to-end success on a single tier depending on which. <i>Model identity is a parameter of the result.</i>
What it was given	Which document, which revision , which section, and the prompt version. Without the revision, nobody can tell whether the property predates the change that made it wrong.
Sampling settings, and how many runs were kept	Sampling parameters are not a seed in Chapter 9's sense : identical inputs can yield different outputs across runs. And regeneration is not re-checking — a second run agreeing with the first is not corroboration, since both come from the same process with the same blind spots.
Which mechanical checks it passed	And their versions: compilation, signal resolution, antecedent reachability, duplicate detection.
Who accepted it at each gate, and against what text	Not a signature on a batch: a name against an artifact, and the specification sentence they compared it to.
Whether it was edited after generation, and by whom	A hand-edited generated property is a hand-written property with a misleading history.

THE LAST TWO ARE THE ONES TEAMS SKIP AND THE ONES A DEFENSIBLE SIGN-OFF NEEDS

A record of what a *machine* produced is an audit trail of the machine. **A record of what a *person* accepted is an audit trail of the decision — and the decision is what is being reviewed.**

Three questions, fifteen minutes, and a decision you can defend

1. A check on the artifact that is *independent of what produced it*? **A proof passed against the very design the artifact came from is not a check.**

2. A *baseline you already own* and can re-measure on your own data? **A vendor benchmark is not a baseline; your own last quarter is.**

3. Does acceptance need a judgment about intent that *no run can settle*? Then **the flow's throughput is that judgment's throughput.**

ADOPT NOW

Where the first two hold and the third does not. **Regression ranking and selection** — the baseline is your own runtime, the check your own coverage database — with slide 23's constraint in the same breath. **Supervised triage classification** where a labelled failure history already exists, with a published agreement rate re-measured on a schedule. **Generated scaffolding** wherever a mechanical diff against a machine-readable source decides correctness.

PILOT

In a *non-gating tier*. **Property generation from a specification**: on a block whose specification is unusually precise, with reviewing capacity budgeted first, and with a statistic of your own — *what fraction survived the intent gate* — from the first batch. **Counterexample debug assistance** is the other good pilot: its output is a hypothesis a human was going to form anyway, and its fix can be applied and the proof re-run, *a check the assistant does not control*.

WAIT

On flows whose sign-off artifact is **the flow's own report**. As of a 2026 survey, reinforcement-learning systems have reached industrial tape-out, and **no language-model-driven autonomous agent had yet demonstrated a complete, human-free tape-out of an industrial chip**. The survey also names the reading error to avoid — *the unit-test fallacy*, reading module-level pass rates as evidence about system-level capability. **Every headline figure here was measured on a block, a protocol or a curated benchmark. None on an SoC.**

*SAY One governance failure cannot be corrected afterwards. A 2023 survey put the industrial concerns under three headings — model generalisation, model scalability and **data governance** — and spelled the third out: an engineer who cannot send her data outside her organisation, and has no machine-learning infrastructure inside it, cannot use a model that requires training on her data. So before a pilot starts, establish what leaves the building — specification, RTL, failure logs, coverage database — in writing, from whoever owns the design's confidentiality rather than from the team running the pilot.*

26

The Road Ahead

Why the binding constraint has moved four times, what shift-left and shift-right genuinely buy, which properties of the problem survive any improvement in tooling — and what the discipline still handles badly.

Everything about the programme improved. The bug got harder to find.

The flagship SoC's eight-channel DMA goes to tape-out: 40-bit physical addressing, descriptor chaining, the same frontend/midend/backend split and the same 4 KB legalization discipline inherited from the reference SoC. **The verification programme around it is unrecognisable.** The 4 KB rule exists as a formal property rather than only as a scoreboard check hoping to meet the right descriptor. The stimulus is a scenario space retargeted from simulation to the prototype instead of a constraint block written for one environment. Regressions gate on every commit, coverage merges across four platforms, and machine time is no longer what this block's team argues about.

WEEK 31 · SOC-1284'S DESCENDANT

The midend mis-legalizes a negative-stride row across a 4 KB frontier again — a different module, wider addresses, **the same arithmetic mistake** — and this time the second half of the split writes a line the host cluster holds Modified. The scoreboard that caught the original sees nothing, because the corrupted bytes never reach memory during the check; they surface at a writeback, later, somewhere else. The team finds it eventually, from a firmware symptom on the prototype: **weeks of debug on a platform where the corrupted line is not visible, for a bug whose ancestor a scoreboard caught in one night.**

WHAT DECIDED THE OUTCOME IN 1284

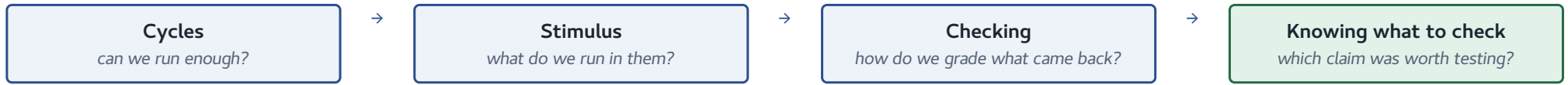
Two hundred and fourteen directed tests passed *honestly*, and a constrained-random run stumbled into the combination. The escape analysis recorded that **the functional coverage model had no cross of the three dimensions**, so the hole was invisible to the team's own metrics.

AND IN THE DESCENDANT

No engine was wrong. The proof was sound — and it was a proof about the *previous generation's* backend, inherited into the new plan as a row already discharged. Nobody wrote the row saying that **a data-integrity check crossing into a coherent domain is a different claim needing different evidence.**

SAY *Both times the binding constraint was the same, and it was not compute, not stimulus, and not checking. It was knowing what to ask. That is the argument of the chapter, and the one thing in it worth carrying.*

A constraint binds when it is what you would spend a doubled budget on



THE BOOK'S ARGUMENT The four-stage migration is **this book's historical read, not a published finding**. The evidence attached to each stage is the corpus's, but each measurement belongs to its own stage and none of it composes: figures from different years, populations and instruments **do not make a trend line**. *The stages share a shape, not a scale.*

Cycles	RTL simulation runs <i>approximately</i> a billion times slower than the target clock. The constraint has not disappeared; it has been <i>relocated by level</i>. On the reference SoC's DMA it has not bound for years. On a flagship firmware run it still does: as of the 2024 data, as frequency scaling plateaus, simulation struggles to keep pace with growing complexity — making acceleration essential <i>for many designs</i> rather than a luxury.
Stimulus	Hand-enumerated tests stop scaling in the low hundreds; the answer was replacing enumeration with <i>description</i> . The tell that this dissolved is the diagnosis attached to its own limits: in 2014, with constrained-random adoption <i>seen to be</i> levelling off, the reason offered was scaling — it works well at IP block or subsystem level and does not scale to the entire SoC integration level. A complaint about a level at which deciding <i>what to generate</i> has become the hard part.
Checking	Generating traffic you cannot grade is a waveform generator, not a testbench. Completion shows in the adoption data as <i>standardisation</i> : SystemVerilog Assertions the predominant assertion language, UVM the predominant methodology. The video codec shows where “dissolved” stops: a bit-exact reference decoder hands you the oracle, but not rate control, latency, or line-buffer back-pressure.

SAY *The first row is the discipline's first lesson about its own constraints: they do not lift, they move down the hierarchy and wait for you at the next level up.*

The evidence is indirect by nature — a question nobody asked leaves no trace in any tool's output

SIGNAL	WHAT WAS MEASURED, AND WHAT IT RULES OUT
The industry outcome	IC/ASIC first-silicon success stood at 14 percent in 2024 — the lowest value recorded in twenty years, and the low point of a declining trend. Separately, on the FPGA side — a different population, a different metric, and not a point on the same curve — 87 percent of projects reported non-trivial bugs escaping into production in 2024. Jointly they rule out the comfortable story that better engines produce better outcomes on their own. The engines got better; adoption climbed; both effectiveness metrics are bad.
What is growing	Not gates: <i>requirements</i> . Complexity growth comes from new layers of design requirements beyond basic functionality — clocking, security, safety and hardware-software interaction, which did not exist years ago. Each layer is a category of question somebody must think to ask before any engine can answer it. A specification defect is precisely a case where the checking machinery was pointed at the wrong claim.
Three corroborating reads	Debug took more of a verification engineer's time than any other activity in 2024. The fastest-growing corner of formal is the applications — 8.7 percent compound annual rate against 5.8 percent for property checking — and an application supplies its own properties from a register description or a connectivity table. And the completeness question has a rigorous answer whose existence makes the point: a property set cannot tell you whether it is enough, so Bormann's criterion asks instead whether the properties pin the design down.

SAY *The accelerator's prefetcher bug escaped 100 percent code coverage entirely; only a cross of weight precision × tensor width class × contention demands the scenario, and that cross exists only if somebody chose those axes. **No engine chooses axes.***

WRG / Siemens EDA 2024 trend reports — IC/ASIC 14% [R2, p. 5], debug p. 7, the 8.7% / 5.8% CAGRs p. 11; FPGA 87% [R1, p. 1]. Double-blind, ±4% at 95%.

Each buys exactly one thing the other cannot

Shift-left means verification starting earlier, against something other than finished RTL. Clock-domain-crossing tools identify domain issues directly on an RTL model at earlier stages, *because the metastability bugs they target cannot be demonstrated on an RTL model in simulation at all*. Virtual platforms let firmware run against functional models before the hardware exists, and are *sometimes* the only option available.

What it genuinely buys is not earlier bug-finding: it is **earlier specification finding**. The most valuable thing that happens when you write an assertion against a document rather than a design is that **the document turns out not to say what everyone assumed**.

ITS LIMIT — THE SENSOR HUB

Its always-on 32 kHz island, wake sources and retention list are architectural decisions, made and reviewed long before the RTL. Real work, done early, and *not simulation of anything*. What it cannot reach is the reference SoC's event-unit escape — a retention register losing its configuration when a wake event races the release of isolation — which surfaced in power-aware gate-level simulation *after* RTL sign-off. **Shift-left moves the claims earlier. It does not move the evidence with them.**

Shift-right means the design continues to be checked after tape-out. Checking real hardware rather than a model is *not a fallback for what verification missed*: it is a planned stage with its own instruments, and the industry pays for it in silicon — in some modern SoCs **over 20 percent of the design real estate** goes to silicon validation. What it buys is the one thing no pre-silicon platform has: **the real device, at real speed, running real software for real durations**.

WHAT IT CANNOT BUY — AND THE CONSTRAINT IS ARITHMETIC, NOT CULTURAL

Observability. Every mechanism writes into a fixed store, so **signals × cycles ≤ buffer**: the flagship's 64 KB trace buffer buys 128 traced signals for 4,096 cycles, or 256 for 2,048 — and *the rectangle is fixed at tape-out*. Which is why the deliverable of a post-silicon debug is not an explanation but **a pre-silicon reproducer**. *Shift-right, done properly, is a machine for moving problems back into the middle of the flow.*

SAY Two more shift-left instruments belong here: portable stimulus, which describes a scenario space once and retargets it; and continuous integration — one 2026 research-programme account reports that running UVM-based verification and FPGA-based checks inside CI increased productivity by making the verification flow transparent from RTL development.

A discipline pulling in both directions is one whose middle is under strain



JUDGMENT, NOT A FINDING Both movements are correct in their own terms. **The failure mode of doing both enthusiastically is a project with an excellent early plan, an excellent lab, and a thin layer of actual verification between them** — the layer where evidence is cheapest to produce and easiest to trust.

SCENARIO · THE FLAGSHIP PROPOSES HALVING THE CROSSBAR'S BLOCK-LEVEL REGRESSION

The argument offered has two halves: its protocol properties are *formally proven*, and its integration behaviour *will be covered in the lab*. **Approach.** The verification lead accepts the first and refuses the second. The proven properties genuinely do retire the protocol rows, so those runs no longer earn their place and go. The rows they do *not* retire are the ones about **arbitration under sustained back-pressure across two managers** — proven only up to a bound, and reachable in the lab but not *diagnosable* there inside the trace buffer. Those seeds stay.

Why. The two ends of the flow retired one class of evidence, and neither of them retired the other. **Cutting by argument row by row is a different act from cutting by percentage — and only the first one can tell you which half you kept.**

SAY The symptom to watch for in a team, not a plan: strong architectural modelling, a well-equipped lab, and block environments staffed by whoever is free. The middle is where both other skills are eventually cashed in — a pre-silicon reproducer is useless to a team that cannot run one.

Would this still be true if every tool got a thousand times better?

Properties of the *problem* survive that test. Properties of the *tooling* do not — and a closing chapter that mixes them ages badly, because a reader ten years out will find half of it quaint and will not know which half. **Six things pass.**

1. The state space is exponential in the number of state bits, and abstraction is the only reply	One DMA channel's 192 bits of architecturally visible state make 2^{192} — about 10^{57} — configurations. A thousandfold faster engine buys you ten bits. The coherent hub makes it permanent: the MESI state of every line in every cache is a product space no tool will enumerate, so the proof runs against a model cut to two caches and one address. <i>Not a workaround awaiting better capacity — it is the technique.</i>
2. Simulation shows the presence of bugs, never their absence	A statement about <i>sampling</i> . No improvement in sampling rate converts a sample into a census.
3. Every practical oracle is partial	Each checks a <i>projection</i> of correctness rather than correctness, and the craft is combining them so their blind spots do not overlap.
4. A proof is a statement about a model, under assumptions, sometimes to a depth	“Formally verified” without those three qualifiers is not a claim. This is the durable form of “ <i>formal doesn't scale</i> ” — which is a tooling fact and will date. The qualifiers will not.
5. A plan is a set of claims, and coverage is a sample of a model you wrote	Recording that a situation <i>occurred</i> and judging whether the design behaved <i>correctly</i> in it are two separate aspects of an environment: they draw on the same monitors, and their fidelities are traded off against one another. So “100 percent” is a statement about your model's own extent that says nothing about its fidelity.
6. Activation, propagation and checking are three independent conditions	A bug found requires all three, and each can fail alone; controllability and observability bound the first two. <i>Which is why observability is designed in a year before it is needed, and why it is a rectangle rather than a dial.</i>

SAY Item 5 carries into Chapter 25 unchanged: a drafted plan is still a set of claims somebody's judgment shaped, and it still cannot see what it cannot represent.

Three that feel permanent and are not — sorting them is what makes the other six worth trusting

“Debug dominates verification time”	A <i>measured share</i> in 2014 and again in 2024 — a robust empirical regularity of the current practice, not a property of the problem , and the most plausible thing on that list for automation to change. Which would move the constraint named fourth rather than retire it : an engine that explains a failure in seconds still does not decide which questions were worth asking.
“Formal is capacity-limited to small blocks”	A statement about engines . The durable form is item 4 on the previous slide — model, assumptions, depth — and those qualifiers survive any capacity improvement.
“Simulation is a billion times slower than silicon”	A measurement of today's tools rather than a law — and treating it as one is how a chapter becomes a period piece.

DURABILITY AUDIT · TWO DMA PLAN ROWS	DURABLE	CONTINGENT ON TODAY'S TOOLING
DMA-F06a — 2D legalization cross at 100%	That the cross is a <i>chosen</i> set of equivalence classes, and that “100%” measures its <i>extent</i> , not its fidelity	That it has four axes and 72 cells; that a solver budget forecloses the maximum-length straddle; that a human wrote the bins
DMA-F06b — p_no_4kb_crossing proven unbounded	That the proof is about a <i>model</i> , under assumptions, at a binding point somebody chose	That it converges unboundedly on this block at this size; that the property text is hand-written

SAY Read the right-hand column as the part of your methodology with an expiry date, and the left as the part that outlives it. Then note where this chapter’s opening bug landed: **the left column of F06b** — the proof is about a model, and the model was the previous generation’s backend.

A profession, on four independent readings

Headcount	Demand for verification engineers has outgrown demand for design engineers since at least 2007 — a 12.5 percent compound annual rate against 3.7 percent through 2014, by which point the <i>mean peak</i> number of verification engineers on a project already exceeded the design count. The growth rates have since converged, but from that inverted baseline — and in <i>some</i> market segments, processors among them, a five-to-one ratio is <i>not unusual</i> .
Boundary	No longer organisational. In 2024, design engineers spent <i>on average</i> 49 percent of their time on verification activities. The profession is defined by the activity rather than by the org chart.
Apparatus	Its own standards: an assertion language, a testbench methodology, a stimulus description language, a coverage interchange format, a power-intent format — and, as of March 2026, a standard for describing an IP's clock- and reset-domain-crossing interface so that different teams' tools can exchange collateral about it. What a field standardises tells you where its work actually is: that list is not about running simulations faster, it is about <i>making claims portable between people</i> .
Theory	State-space arguments, the oracle problem, coverage as sampling, proofs qualified by model and assumption and depth — a body of reasoning that is genuinely its own, and that a good verification engineer uses daily whether or not they can name it.

THE HONEST PITCH, FOR SOMEONE CHOOSING IT

Narrow and strong: **the product is judgment** — and judgment is what the tooling has been least able to absorb through four generations of trying. The work is intellectually severe in a specific way: you spend your time reasoning about what *could* be true rather than about what is. It also carries an unusual professional obligation, **independence of judgment**, which comes down to a willingness to say “*no, this is not the way to do it*” to a designer with more years on the block, or to a schedule that wants the block closed. **A duty rather than a personality trait — and the day it is inconvenient is the day it counts.**

The 49%: 2024 Wilson Research Group IC/ASIC trend report — [R2, p. 7]. The clock- and reset-domain-crossing standard is Accellera's *Standard for IP Abstraction for Clock and Reset Domain Crossing Integration*, v1.0, March 2026 — [S12, pp. 7–8]; its working group ran to 154 members from 24 companies — [D36, slide 58]. The 12.5% / 3.7% headcount growth rates are not from these sources.

... and four things it still handles badly

JUDGMENT, NOT PESSIMISM Every one of the four is a place where an individual can be visibly better than the average.

It does not learn from its escapes	Escape analysis, where it happens at all, is usually a post-mortem slide rather than a <i>change to the next plan and the next sign-off checklist</i> . The mechanism for compounding organisational knowledge exists, is cheap, and is skipped.
It cannot estimate	Debug takes more time than anything else, is unpredictable, and varies significantly between projects — which makes planning a project's effort from the previous project's data a <i>stated management problem, not a solved one</i> . In practice, teams estimate that way regardless. Three quarters of IC/ASIC projects were behind their original schedule in 2024 , against a historical average of about two thirds.
It rarely measures its own environments	Every instrument in this book measures the design. Mutation is the one technique that answers “ <i>would this check have caught a change?</i> ” — and <i>in practice</i> it runs on a small fraction of blocks. A checker nobody has seen fail is a decoration , and that is true at the scale of a whole environment, not just of a single assertion.
It has no common language for “how verified is this?”	A functional sign-off aggregates coverage and plan rows. A safety argument aggregates fault classification against per-level metric thresholds. A security result is a list of named claims and named gaps <i>with no percentage available at all</i> , and there will not be one soon. The three are not commensurable, they arrive at the same tape-out review, and the discipline has no accepted way to state a combined position. An open problem, not a solved one presented badly.

SAY The cheap countermeasure to the first item and the fourth: keep an explicit register of what the current programme is not verifying — the gaps, not just the waivers — because a named gap is reviewable and an unnamed one is a surprise. It has no metric attached, which is why it loses its budget argument to things that do.

Verification does not produce a correct design. It produces a *true sentence about one*.

What it produces is this: *this is what we established, by this method, under these assumptions, and here is what we did not establish*. Sign-off is the moment that sentence is spoken aloud and somebody's name goes under it.

EVERYTHING IN THIS BOOK IS MACHINERY FOR MAKING THAT SENTENCE MEAN MORE	
A plan	the sentence written down in advance
Coverage	evidence that the sentence's scope was actually visited
A scoreboard	the part that could have made it false
An assertion	a clause of it, checked continuously
A waiver	the sentence being honest about its own edges
A proof	a clause with no exceptions inside a named model
An escape analysis	the sentence being repaired after it turned out to be too generous

SOC-1284 IS WHAT A FALSE SENTENCE LOOKS LIKE FROM THE INSIDE

Two hundred and fourteen directed tests passed, honestly. The dashboard was green. The coverage report was accurate about everything it measured. **The sentence all of that supported was *the DMA's 2D legalization is verified* — which was not true, and no instrument in the project could say so.**

The question to bring to every review, every plan and every green regression for the rest of your career is not *are we done?*
It is: what exactly are we claiming, and what would have to be true for that claim to be false?
If you can answer the second half, you are verifying. If you cannot, you are counting.

SAY Close the loop to slide 6: Chapter 24 asks an owner to state what would have to be true for their answer to be wrong — offered there as the author's read rather than a published rule — and Chapter 26 asks the same thing of a whole sign-off.